

# V-FINDERS Framework: A Project Manager's Guide to Complete Requirements Discovery

## Introduction

V-FINDERS is a structured requirements discovery framework for planning, scoping, and delivering complex business, operational, and technology projects. It is designed to help project managers, product owners, analysts, implementation teams, and stakeholders identify the full range of requirements needed for a system or process change to succeed.

The framework evolved from the FRIENDS and FINDERS models, which organized requirements around Forms, Integration, Non-Technical needs, Data, Enhancements, Reports, and Set-Up. V-FINDERS adds a critical first category: **Views & Experience**.

This addition matters because successful systems are not judged only by whether they store data, move data, or generate reports. They are judged by whether users can see the right information, in the right context, at the right moment, with minimal cognitive burden.

**V-FINDERS starts with the screen because the screen is where value becomes visible.**

---

## What Is V-FINDERS?

**V-FINDERS** is an acronym representing eight major requirement categories:

- **V** — Views & Experience
- **F** — Forms
- **I** — Integration / Interfaces
- **N** — Non-Technical
- **D** — Data
- **E** — Enhancements / Extensions
- **R** — Reports
- **S** — Set-Up

Together, these categories provide a practical checklist for answering one central question:

What do we need to account for so this project works in the real world?

V-FINDERS ensures that every system requirement is traced from user experience, through data capture and exchange, into operational support, reporting, and infrastructure readiness.

---

## Why Views Come First

Many projects begin by documenting forms, integrations, reports, data migration, and infrastructure. These are essential, but they do not fully describe how users experience the system during real work.

A system may have accurate data, stable interfaces, and complete reports, yet still fail if users cannot easily see, understand, trust, and act on information when it matters.

Views come first because they represent the operational decision surface. This is where users monitor work, identify exceptions, assess status, navigate records, respond to alerts, and complete tasks.

Forms collect data.

Interfaces move data.

Data structures store data.

Reports summarize data.

**Views help users act on data.**

In healthcare, this distinction is especially important. A medication list, abnormal lab highlight, patient banner, allergy warning, triage queue, order status, bed board, or discharge task list is not merely a report. It is a live operational view. Poorly designed, it increases cognitive load and risk. Properly designed, it supports timely, safe, and efficient decisions.

---

## The Eight Components of V-FINDERS

### V — Views & Experience

#### What It Covers

Views & Experience covers the live user interface, screen layouts, dashboards, worklists, navigation paths, alerts, visual hierarchy, and interaction patterns through which users consume and act on information inside the system.

This category is not limited to charts or dashboards. It includes every screen, queue, list, indicator, message, and interaction pattern that affects how users understand the current state of work.

#### Key Question

**How do users see, interpret, navigate, and act on data?**

#### Key Considerations

- What information must be immediately visible to the user?
- What information can be secondary or hidden behind drill-downs?
- What should never be buried behind excessive clicks?
- Do different roles need different views of the same data?

- Are worklists, queues, dashboards, or boards required?
- How are abnormal values, overdue tasks, pending approvals, blocked workflows, or exceptions displayed?
- Which alerts are interruptive, passive, escalated, suppressed, or color-coded?
- How is alert fatigue prevented?
- How much data can safely appear on one screen without overwhelming the user?
- Are users shown trends, comparisons, histories, or point-in-time values?
- How many clicks does it take to complete high-frequency or high-risk tasks?
- Are font size, terminology, color contrast, and layout appropriate for the environment?
- Will the view be used on desktop, tablet, mobile, workstation-on-wheels, kiosk, or shared terminal?
- What happens when data is missing, delayed, conflicting, or unavailable?

### **Example Deliverables**

- UI/UX requirements catalog
- Screen inventory
- Wireframes and mockups
- User journey maps
- Role-based dashboard specifications
- Worklist and queue design
- Alert and notification matrix
- Data visualization standards
- Click-path analysis
- Usability testing plan
- Accessibility checklist
- Prototype review and sign-off documentation

### **Definition of Done**

Key screens, dashboards, worklists, alerts, and navigation paths are documented, prototyped where appropriate, and validated with representative users using realistic scenarios.

---

## **F — Forms**

### **What It Covers**

Forms cover all points where information is entered, captured, submitted, approved, or updated. This includes paper forms, digital forms, embedded system forms, intake screens, questionnaires, checklists, templates, and structured data entry workflows.

Forms are how information enters the system.

### **Key Question**

**How does information enter the system?**

## Key Considerations

- What forms are currently used?
- Which paper forms need to be digitized?
- Which system forms already exist and which must be configured or built?
- What fields are mandatory, optional, calculated, or conditional?
- What validation rules are required?
- What approvals or routing steps are involved?
- Who can create, edit, submit, approve, reject, or archive the form?
- What data from the form feeds views, reports, integrations, or downstream processes?
- What happens if the form is incomplete, duplicated, rejected, or submitted in error?

## Example Deliverables

- Forms inventory
- Forms analysis document
- Digital form specifications
- Field-level requirements
- Validation rules
- Approval workflow diagrams
- Form-to-data mapping
- Forms testing and validation plan

## Definition of Done

Data entry fields, validation rules, approval flows, ownership, downstream usage, and exception handling are documented and approved.

---

# I — Integration / Interfaces

## What It Covers

Integration / Interfaces covers all system-to-system connections, data exchanges, communication protocols, interface engines, APIs, file transfers, middleware, and third-party connections.

Integration is how systems talk to each other.

## Key Question

**How do systems exchange data?**

## Key Considerations

- Which systems need to send or receive data?
- Is the flow inbound, outbound, or bidirectional?
- What data is exchanged?

- What triggers the exchange?
- Is the exchange real-time, near-real-time, scheduled, batch, or manual?
- What standards or protocols are used?
- What transformations, mappings, or code translations are required?
- What happens when an interface fails?
- Who monitors interface errors?
- Who owns issue resolution across systems?
- What audit logs are required?
- What are the expected data volumes and peak loads?

### **Example Deliverables**

- Integration architecture diagram
- Interface inventory
- Interface specification document
- Source-to-target mapping
- Transformation rules
- Error handling and retry logic
- Interface monitoring plan
- Interface testing and validation plan
- Operational support model

### **Definition of Done**

Source, destination, trigger, payload, frequency, protocol, transformation rules, error handling, monitoring, ownership, and test cases are defined and approved.

---

## **N — Non-Technical**

### **What It Covers**

Non-Technical covers the human, procedural, organizational, policy, training, governance, and support requirements needed to make the solution usable and sustainable.

This category includes everything that supports the system but is not the system itself.

### **Key Question**

**What people, process, policy, training, and support needs exist?**

### **Key Considerations**

- What policies and procedures must change?
- What standard operating procedures are required?
- What manual processes remain outside the system?
- What user roles and responsibilities must be defined?

- What training is needed by role?
- What job aids, quick reference guides, or support scripts are needed?
- What change management activities are required?
- What support model will be used after go-live?
- What is the escalation path for incidents and defects?
- Who owns business process governance after implementation?

### **Example Deliverables**

- Standard operating procedures
- Training materials
- Job aids and quick reference guides
- Help desk scripts
- Support model
- Escalation matrix
- Change management plan
- Communications plan
- Governance model
- Business readiness checklist

### **Definition of Done**

SOPs, training, support model, escalation paths, business ownership, and operational readiness requirements are documented and accepted by the responsible teams.

---

## **D — Data**

### **What It Covers**

Data covers all information assets, including data definitions, data models, master data, reference data, migrated data, data quality rules, ownership, retention, governance, and reconciliation.

Data is the foundation of the system. If the data is wrong, incomplete, duplicated, ambiguous, or poorly governed, every view, form, interface, report, and enhancement built on top of it becomes fragile.

### **Key Question**

**What data is created, migrated, governed, validated, and maintained?**

### **Key Considerations**

- What master data is required?
- What reference data or code sets are needed?
- What data must be migrated from legacy systems?
- What data must be cleansed before migration?
- What are the authoritative sources for each data element?

- What are the validation rules and quality thresholds?
- Who owns each major data domain?
- What data retention and archiving rules apply?
- What reconciliation is required after migration or integration?
- What data definitions must be standardized across teams?

### **Example Deliverables**

- Data dictionary
- Data model
- Master data inventory
- Reference data catalog
- Data migration strategy
- Source-to-target mapping
- Data quality assessment
- Data cleansing plan
- Reconciliation plan
- Data governance model

### **Definition of Done**

Data definitions, ownership, migration scope, quality rules, validation approach, reconciliation method, and governance responsibilities are documented and approved.

---

## **E — Enhancements / Extensions**

### **What It Covers**

Enhancements / Extensions cover system modifications, configuration changes, custom logic, automation, workflow extensions, custom modules, decision support, rules engines, or other changes beyond standard functionality.

This category captures the gap between what the system does out of the box and what the organization needs it to do.

### **Key Question**

**What must be customized, extended, automated, or newly built?**

### **Key Considerations**

- What gaps exist between standard functionality and business requirements?
- Can the gap be solved through configuration rather than customization?
- Is custom development truly necessary?
- What business rules must be automated?
- What workflow logic is required?

- What dependencies exist with forms, data, views, reports, or integrations?
- What risks are introduced by customization?
- How will enhancements be tested, maintained, and upgraded?
- Who owns the enhancement after go-live?

## **Example Deliverables**

- Gap analysis
- Enhancement requirements
- Functional specifications
- Technical specifications
- Workflow logic documentation
- Business rules catalog
- Custom module design
- Build plan
- Test cases
- Acceptance criteria
- Maintenance plan

## **Definition of Done**

Gaps, custom logic, build scope, dependencies, risks, test cases, acceptance criteria, and ownership are documented and approved.

---

# **R — Reports**

## **What It Covers**

Reports cover all mechanisms for extracting, summarizing, distributing, printing, exporting, or analyzing information. This includes operational reports, management reports, compliance reports, financial reports, regulatory reports, scheduled extracts, and ad hoc analysis.

Reports are how information leaves the system in a structured, reviewable, shareable, or analytical format.

## **Key Question**

**How is information extracted, summarized, distributed, or analyzed?**

## **Key Considerations**

- Who is the audience for the report?
- What decision or obligation does the report support?
- What data sources are required?
- What filters, groupings, calculations, and formats are needed?
- Is the report operational, analytical, compliance-related, financial, or executive-facing?
- Is the report scheduled, on-demand, printed, exported, or embedded?



- What access controls apply?
- How is report accuracy validated?
- What happens when the report conflicts with another source?
- Can an existing report meet the need, or is a custom report required?

## **Example Deliverables**

- Report inventory
- Report requirements catalog
- Report mockups
- Data source mapping
- Calculation logic
- Distribution matrix
- Access control matrix
- Report validation plan
- Report retirement list

## **Definition of Done**

Audience, purpose, data source, logic, format, frequency, distribution, access control, and validation approach are documented and approved.

---

# **S — Set-Up**

## **What It Covers**

Set-Up covers the technical and physical environment required to support the system. This includes infrastructure, environments, hardware, devices, printers, scanners, network connectivity, access, security configuration, deployment readiness, and operational monitoring.

Set-Up is what allows the solution to run reliably in the real world.

## **Key Question**

**What infrastructure, configuration, hardware, network, and environment support is required?**

## **Key Considerations**

- What application, database, and integration environments are required?
- What servers, cloud services, or hosting arrangements are needed?
- What end-user devices are required?
- Are specialized peripherals needed, such as barcode scanners, label printers, biometric devices, card readers, or signature pads?
- What network bandwidth, segmentation, firewall, VPN, or wireless coverage is required?
- What identity and access management setup is needed?
- What backup, recovery, monitoring, and downtime procedures are required?

- What deployment environments are needed for development, testing, training, staging, and production?
- What procurement lead times could affect the project schedule?

### Example Deliverables

- Infrastructure architecture diagram
- Environment plan
- Hardware and device inventory
- Procurement list
- Network requirements
- Access and role configuration plan
- Deployment plan
- Backup and recovery plan
- Monitoring plan
- Technical readiness checklist

### Definition of Done

Required environments, devices, infrastructure, access, network, deployment, monitoring, backup, and support needs are documented and ready for implementation.

---

## Distinguishing Views, Forms, and Reports

Views, Forms, and Reports are related, but they are not interchangeable.

| Category       | Purpose                                                                         |
|----------------|---------------------------------------------------------------------------------|
| <b>Views</b>   | How users see, interpret, navigate, and act on information                      |
| <b>Forms</b>   | How users capture, submit, or update information                                |
| <b>Reports</b> | How users extract, summarize, distribute, print, export, or analyze information |

Use this simple rule:

If the user is making an operational decision inside the system, it belongs under **Views**.

If the user is entering information, it belongs under **Forms**.

If the user is reviewing, exporting, printing, distributing, or analyzing information after the fact, it belongs under **Reports**.

This distinction prevents the common mistake of treating every screen as a form or every dashboard as a report.

---

## Cross-Cutting Lenses

Some requirements are too important to be trapped inside a single category. They should be applied across all V-FINDERS categories.

### Security, Privacy, Compliance, and Risk

Security and compliance are not separate afterthoughts. They must be evaluated across every requirement category.

| Category             | Security / Compliance Question                                               |
|----------------------|------------------------------------------------------------------------------|
| <b>Views</b>         | Who is allowed to see this information?                                      |
| <b>Forms</b>         | What sensitive data is collected, and how is it protected?                   |
| <b>Integration</b>   | Is data encrypted, authenticated, logged, and auditable in transit?          |
| <b>Non-Technical</b> | What policies, procedures, and accountability structures are required?       |
| <b>Data</b>          | What privacy, retention, consent, ownership, and quality rules apply?        |
| <b>Enhancements</b>  | Does customization introduce operational, security, or compliance risk?      |
| <b>Reports</b>       | Can exported or printed data be misused, overshared, or retained improperly? |
| <b>Set-Up</b>        | Is the infrastructure hardened, monitored, backed up, and access-controlled? |

### Adoption and Change Readiness

A requirement is not complete until the user can understand it, perform it, support it, and recover from errors.

Adoption should be considered across all categories:

- Can users understand the screen?
- Can users complete the form correctly?
- Can support teams troubleshoot interface failures?
- Can users trust the data?
- Can teams maintain custom enhancements?
- Can managers interpret the reports?
- Can operations support the infrastructure?

### Data Quality

Data quality affects every category. Poor data weakens views, forms, integrations, reports, enhancements, and decision-making.

Ask:

- Is the data complete?
- Is it accurate?
- Is it timely?
- Is it consistently defined?
- Is it owned by someone?
- Is it trusted by users?

## **Operational and Clinical Risk**

For high-risk environments, especially healthcare, requirements should be reviewed for potential safety, workflow, and operational impact.

Ask:

- Could this design delay action?
  - Could it hide critical information?
  - Could it create duplicate work?
  - Could it increase alert fatigue?
  - Could it lead to wrong-user, wrong-patient, wrong-order, wrong-product, or wrong-location errors?
  - Could downtime or missing data disrupt essential operations?
- 

## **How to Apply V-FINDERS**

### **1. Use It During Initial Scoping**

At project initiation, walk through each V-FINDERS category and identify what may be in scope. This creates a broad but structured view of the project before detailed requirements work begins.

Ask:

- What views will users need?
- What forms must be created or changed?
- What integrations are required?
- What non-technical changes are needed?
- What data is involved?
- What enhancements are required?
- What reports are needed?
- What setup work must be completed?

### **2. Structure Requirements Workshops Around the Categories**

Use V-FINDERS to organize stakeholder interviews and workshops. Different stakeholders will know different parts of the framework.

For example:

- Frontline users are often strongest on Views and Forms.
- Technical teams are strongest on Integration, Data, and Set-Up.
- Managers are often strongest on Reports and operational metrics.
- Training, support, and governance teams are strongest on Non-Technical requirements.

### **3. Build a Requirements Traceability Matrix**

Each requirement should be linked to a category, business objective, owner, priority, and status.

Suggested fields:

- Requirement ID
- V-FINDERS category
- Requirement description
- Business objective
- Priority
- Owner
- Dependencies
- Acceptance criteria
- Status

### **4. Map Cross-Category Dependencies**

Requirements rarely stay inside one category.

Examples:

- A View depends on Data definitions.
- A Report depends on Forms capturing the right fields.
- An Integration depends on source system data quality.
- An Enhancement may require new Set-Up work.
- A Form may trigger a workflow that appears in a View.

Document these dependencies early. This prevents teams from designing in silos.

### **5. Use V-FINDERS Iteratively**

V-FINDERS can be used in waterfall, agile, or hybrid delivery models.

In waterfall projects, use it during formal requirements gathering and design.

In agile projects, use it during backlog refinement, sprint planning, sprint review, and release planning.

In hybrid projects, use it for upfront scope framing and then refine each category incrementally by release.

## 6. Validate With Realistic Scenarios

Do not validate requirements only as isolated statements. Validate them through realistic user scenarios.

Examples:

- A user receives incomplete information.
- An interface fails.
- A duplicate record appears.
- An abnormal result must be acted on.
- A report number does not match another source.
- A device or printer is unavailable.
- A user lacks the right access.

Normal workflows are easy. Exceptions reveal whether the design will survive production.

---

## Common Pitfalls to Avoid

### 1. Treating Views as Reports

A dashboard, worklist, or operational screen is not automatically a report. Reports are typically used for review, analysis, export, distribution, or compliance. Views are used for live action.

**Watch out for:** Users having to run reports to do daily operational work.

**Advice:** If a user needs the information to act now, design it as a View.

### 2. Starting With the Database Instead of the User

Data models are critical, but they should not be designed in isolation from user workflows.

**Watch out for:** Technically clean data structures that produce confusing screens or unsafe workflows.

**Advice:** Start with the decisions users need to make, then trace backward to the data needed to support those decisions.

### 3. Focusing Only on Integration and Enhancements

Technical teams often gravitate toward integrations and custom build work. Important, yes. Sufficient, no.

**Watch out for:** Interfaces and custom modules receiving detailed attention while training, support, forms, reports, and user experience are treated lightly.

**Advice:** Use all eight V-FINDERS categories during scope and readiness reviews.

## 4. Treating Data as an Afterthought

Bad data will damage every part of the project.

**Watch out for:** Migration and data cleansing being underestimated until late testing.

**Advice:** Start data profiling, ownership discussions, and reconciliation planning early.

## 5. Assuming Standard Reports Are Enough

Out-of-the-box reports rarely satisfy all operational, financial, compliance, and management needs.

**Watch out for:** Stakeholders discovering reporting gaps after go-live.

**Advice:** Build a report inventory and validate report purpose, audience, source, logic, and distribution.

## 6. Ignoring Non-Technical Readiness

A technically functional system can still fail if users are not trained, SOPs are missing, support is unclear, or process ownership is unresolved.

**Watch out for:** Go-live plans that focus on deployment but not adoption.

**Advice:** Treat training, support, SOPs, and governance as formal requirements.

## 7. Leaving Set-Up Too Late

Infrastructure, devices, printers, scanners, network access, and environment setup often have long lead times.

**Watch out for:** Build completion being blocked by hardware, network, access, or environment issues.

**Advice:** Start Set-Up discovery early, especially when procurement or facility constraints are involved.

## 8. Ignoring Exception States

Most requirements workshops focus on the happy path. Production failures usually happen in the exception paths.

**Watch out for:** Designs that do not explain what users see or do when data is missing, delayed, conflicting, duplicated, rejected, or unavailable.

**Advice:** For every major workflow, ask: “What happens when this goes wrong?”

## 9. Confusing Completeness With Priority

V-FINDERS helps identify the full scope of requirements. It does not automatically decide what should be delivered first.

**Watch out for:** Treating every requirement as equally important.

**Advice:** After using V-FINDERS for completeness, apply prioritization methods such as MoSCoW, value versus effort, risk ranking, or MVP definition.

## 10. Letting Category Debates Slow Progress

Some requirements may fit in more than one category.

**Watch out for:** Teams spending more time debating classification than understanding the requirement.

**Advice:** Put the requirement where it best fits, cross-reference related categories, and keep moving.

---

## V-FINDERS Quick Reference

| Category                             | Focus Area                                                         | Key Question                                                     |
|--------------------------------------|--------------------------------------------------------------------|------------------------------------------------------------------|
| <b>V — Views &amp; Experience</b>    | Live data presentation and interaction                             | How do users see, interpret, navigate, and act on data?          |
| <b>F — Forms</b>                     | Data entry and submission                                          | How does information enter the system?                           |
| <b>I — Integration / Interfaces</b>  | System connections and data exchange                               | How do systems exchange data?                                    |
| <b>N — Non-Technical</b>             | People, process, policy, training, and support                     | What operational support is needed?                              |
| <b>D — Data</b>                      | Data structure, quality, migration, and governance                 | What data is created, migrated, governed, and maintained?        |
| <b>E — Enhancements / Extensions</b> | Customization, automation, and added functionality                 | What must be customized, extended, or newly built?               |
| <b>R — Reports</b>                   | Extracted, summarized, distributed, or analytical outputs          | How is information reviewed, exported, distributed, or analyzed? |
| <b>S — Set-Up</b>                    | Infrastructure, configuration, hardware, network, and environments | What technical foundation is required?                           |

---



## Definition of Done Summary

| Category                         | Definition of Done                                                                                        |
|----------------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Views &amp; Experience</b>    | Key screens, dashboards, worklists, alerts, and navigation paths are validated with representative users. |
| <b>Forms</b>                     | Data entry fields, validation rules, approvals, and submission flows are documented.                      |
| <b>Integration / Interfaces</b>  | Source, destination, trigger, payload, protocol, frequency, error handling, and ownership are defined.    |
| <b>Non-Technical</b>             | SOPs, training, support model, escalation paths, and operational ownership are ready.                     |
| <b>Data</b>                      | Data model, migration, quality rules, ownership, and reconciliation approach are defined.                 |
| <b>Enhancements / Extensions</b> | Gaps, custom logic, build scope, test cases, and acceptance criteria are approved.                        |
| <b>Reports</b>                   | Audience, purpose, data source, format, frequency, distribution, and access are defined.                  |
| <b>Set-Up</b>                    | Environments, devices, infrastructure, access, network, deployment, and monitoring needs are ready.       |

## Final Thought

V-FINDERS is more than a requirements checklist. It is a safeguard against incomplete thinking.

It reminds teams that successful systems are not just built, integrated, configured, or reported on. They are experienced by people under real operational conditions.

A complete system must allow users to see what matters, enter what is needed, exchange data reliably, follow clear processes, trust the information, extend the system where necessary, report with confidence, and run on stable infrastructure.

The practical test is simple:

Can the right user see the right information, act correctly, and trust the system when the work matters most?

If the answer is yes across all eight categories, the project is no longer just technically complete. It is operationally ready.